SPACE ORBITERS

Practical Exam Project Report

Dockerized 3-Tier Application • PostgreSQL • Docker Compose • AWS EC2

Structured implementation evidence with additional production deployment enhancements

Repository	github.com/shashankcodes-10/SPACE_ORBITERS
Deployment	space.shashankpipal.in
Database	PostgreSQL 16
Deployment Platform	AWS EC2 (Ubuntu)

Prepared as a structured evidence report

Report Overview

This report presents the implementation in a step-by-step format. The main sections document the practical project work, while DNS, SSL/HTTPS, Nginx reverse proxying, and Jenkins are presented separately as additional production-oriented enhancements.

Core implementation	Project structure, Git practices, architecture, Dockerization, image optimization, Compose, networking, persistence, and EC2 deployment.
Additional enhancements	Elastic IP, GoDaddy DNS, Nginx reverse proxy, Let's Encrypt HTTPS, and Jenkins CI/CD/DevSecOps setup.
Evidence format	Screenshots are kept in the supplied sequence except where specific screenshots were intentionally moved to the topic they prove.

STEP 1

Project Foundation & Git Practices

The project was organized as a full-stack space exploration application and maintained through Git/GitHub with incremental, meaningful changes.

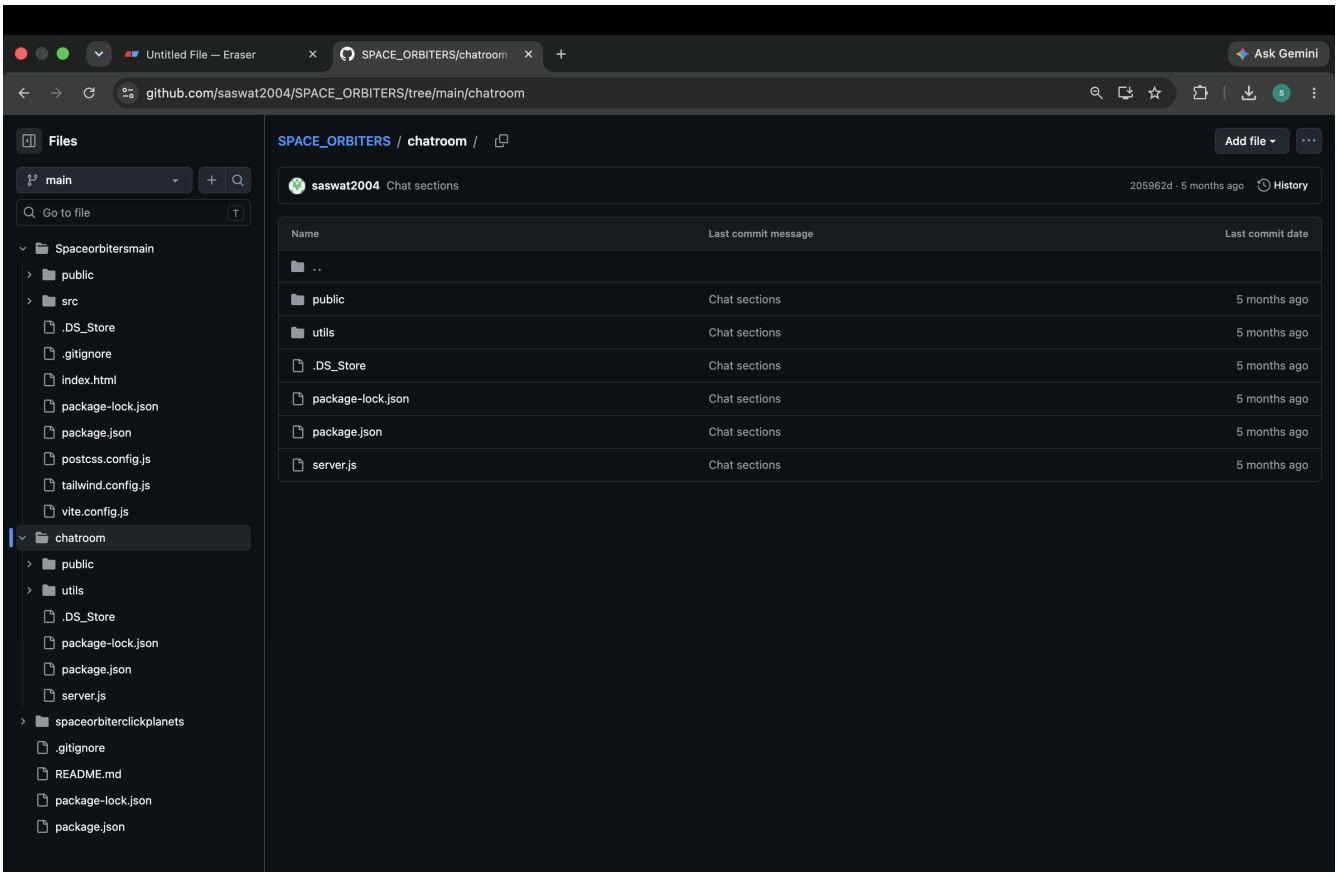

Figure 1 — GitHub repository and project contents.

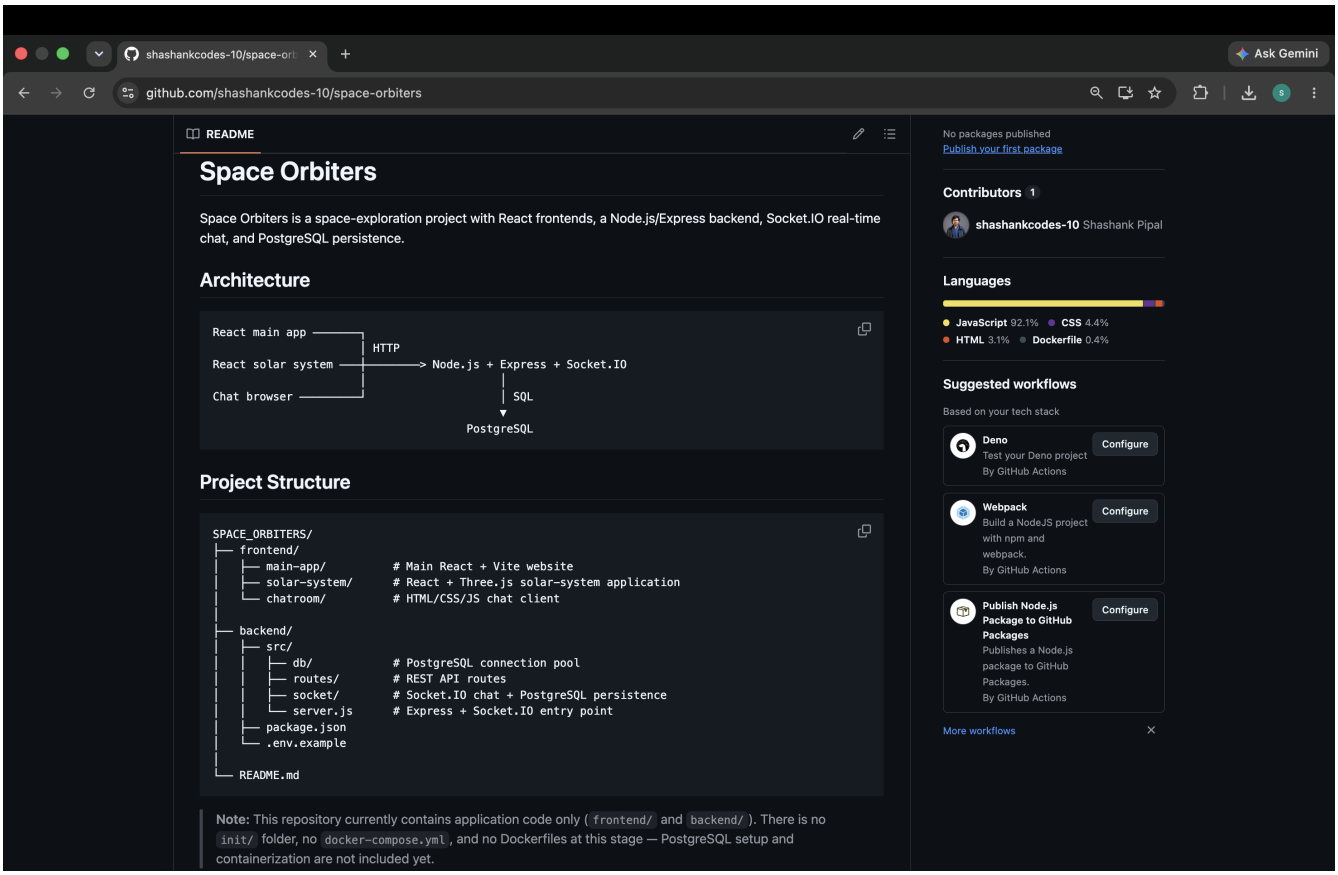

Figure 2 — Repository README and project structure.

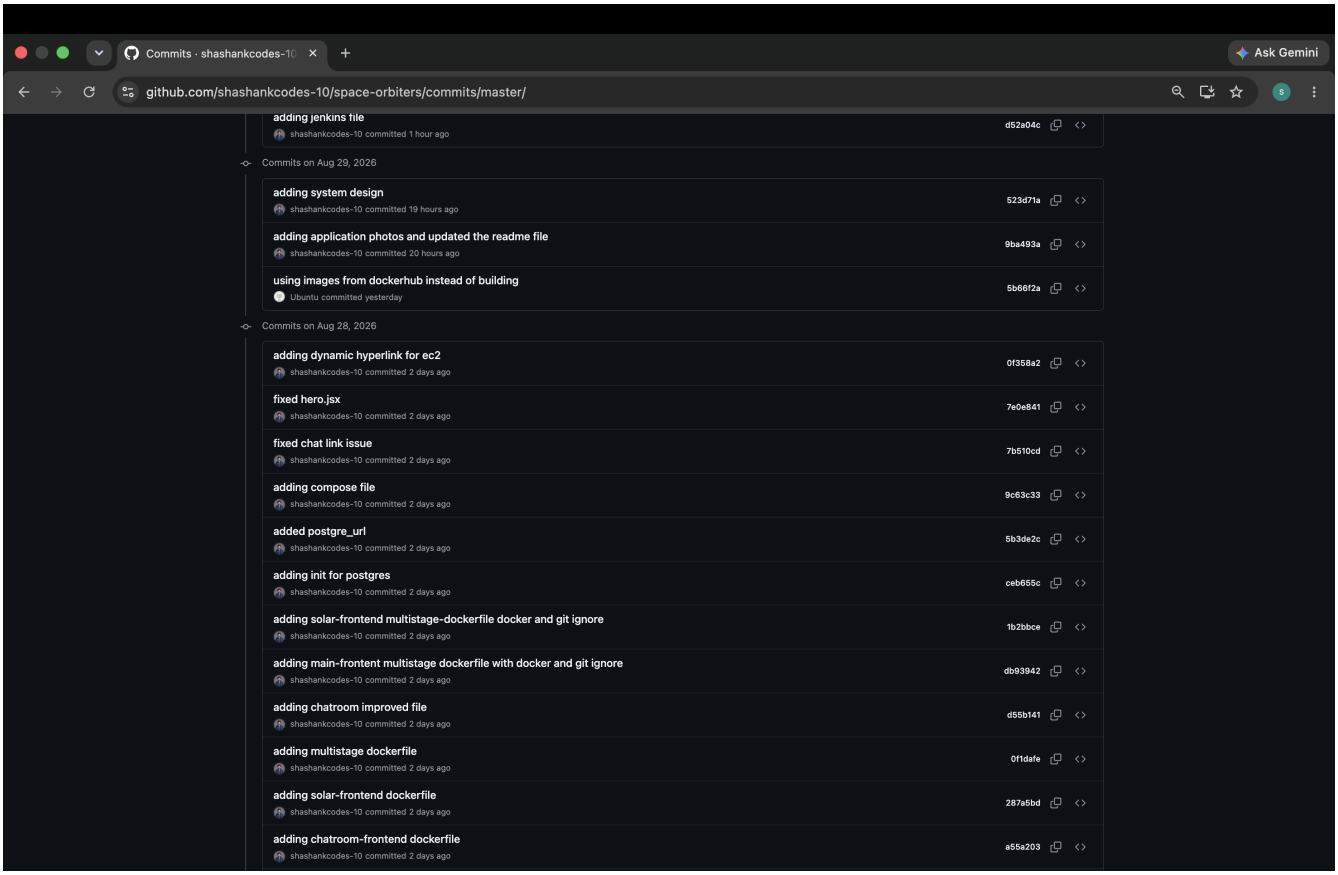

Figure 3 — Git commit history showing incremental project changes, fixes, Docker work, PostgreSQL work, and deployment-related commits.

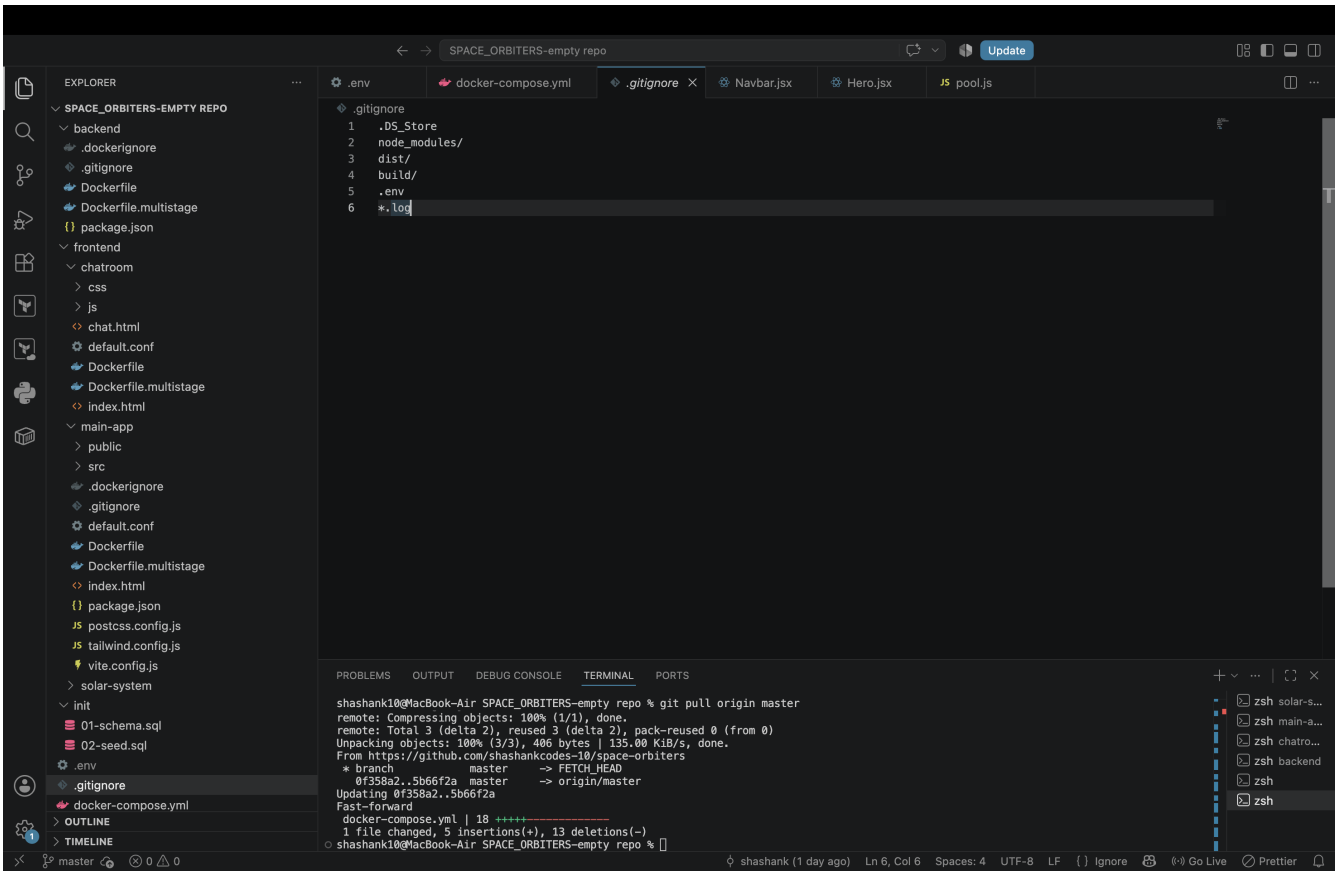

Figure 4 — .gitignore and repository structure in the development workspace.

STEP 2

System Design & Architecture

The architecture uses a 3-tier application model with presentation, application, and data responsibilities separated across containers.

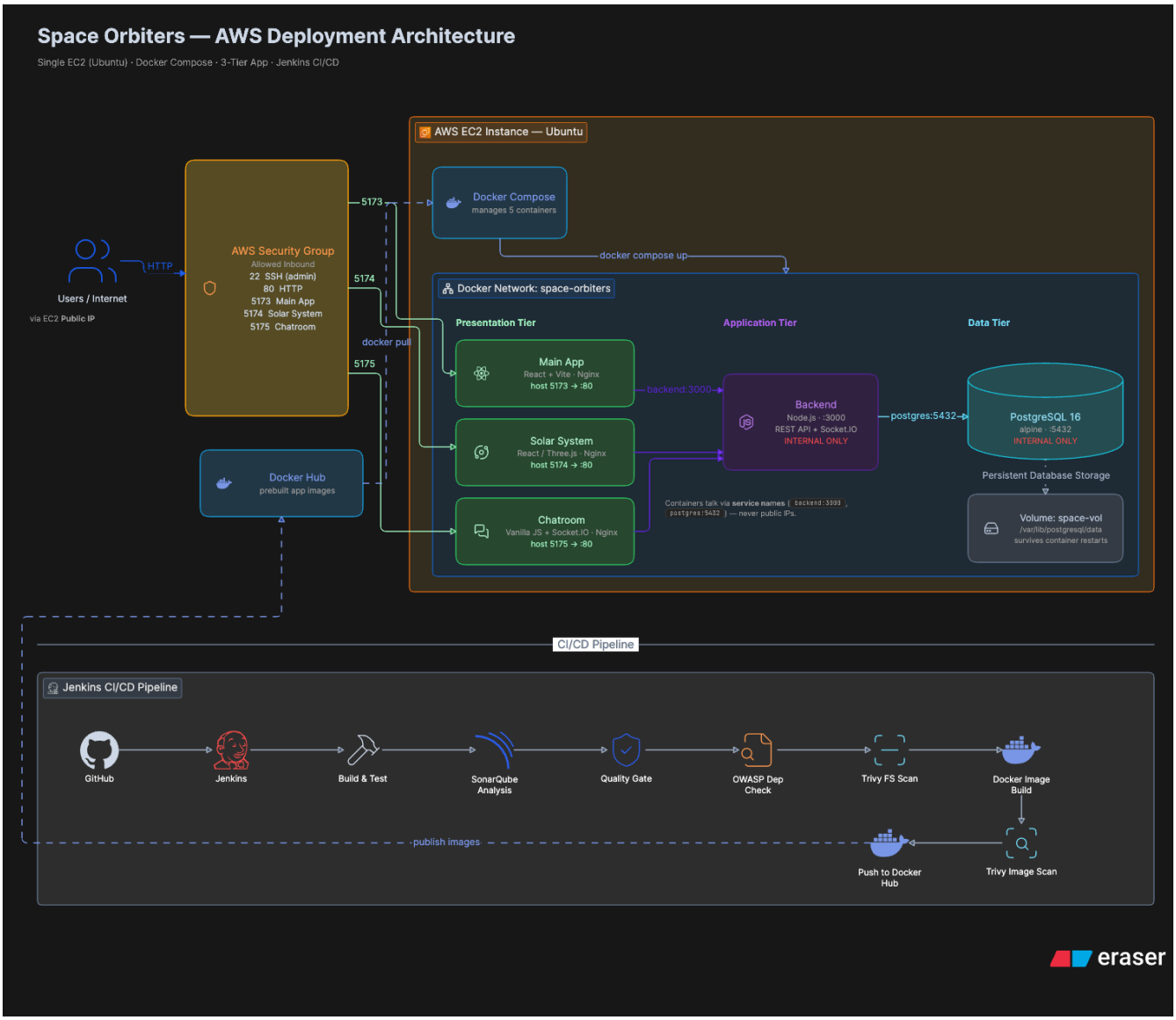

Figure 5 — Space Orbiters AWS deployment architecture and CI/CD design.

STEP 3

Dockerization

Each application component was containerized. Multi-stage Dockerfiles were introduced for the backend and frontend applications to produce leaner runtime images.

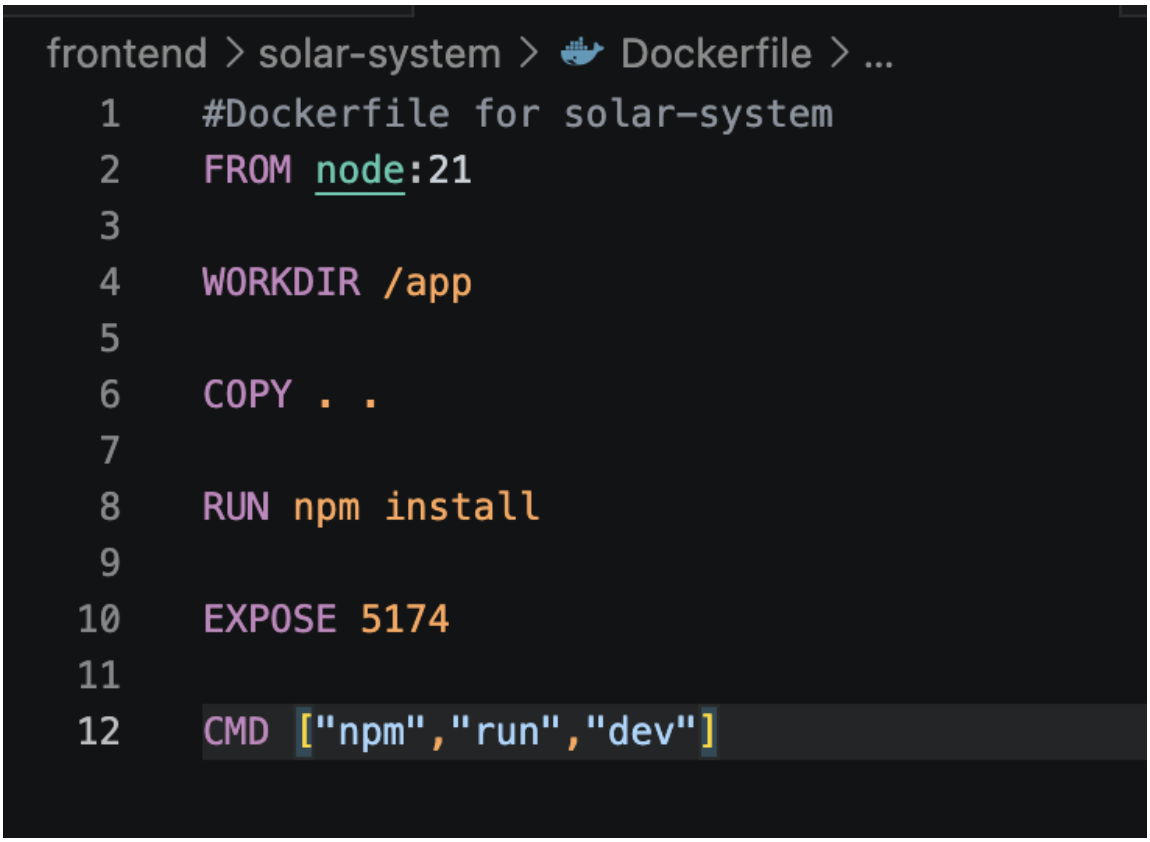

Figure 6 — Solar System frontend Dockerfile.

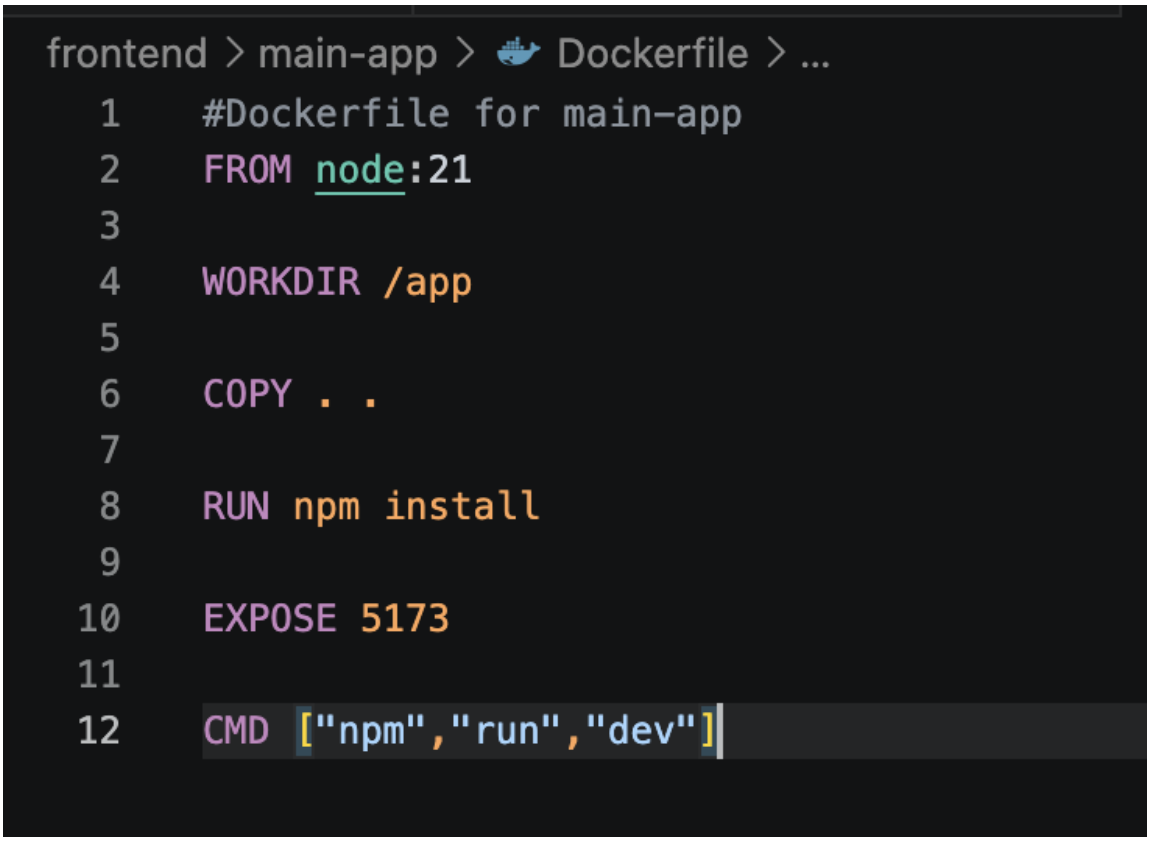

Figure 7 — Main App Dockerfile.

Figure 8 — Chatroom Dockerfile.

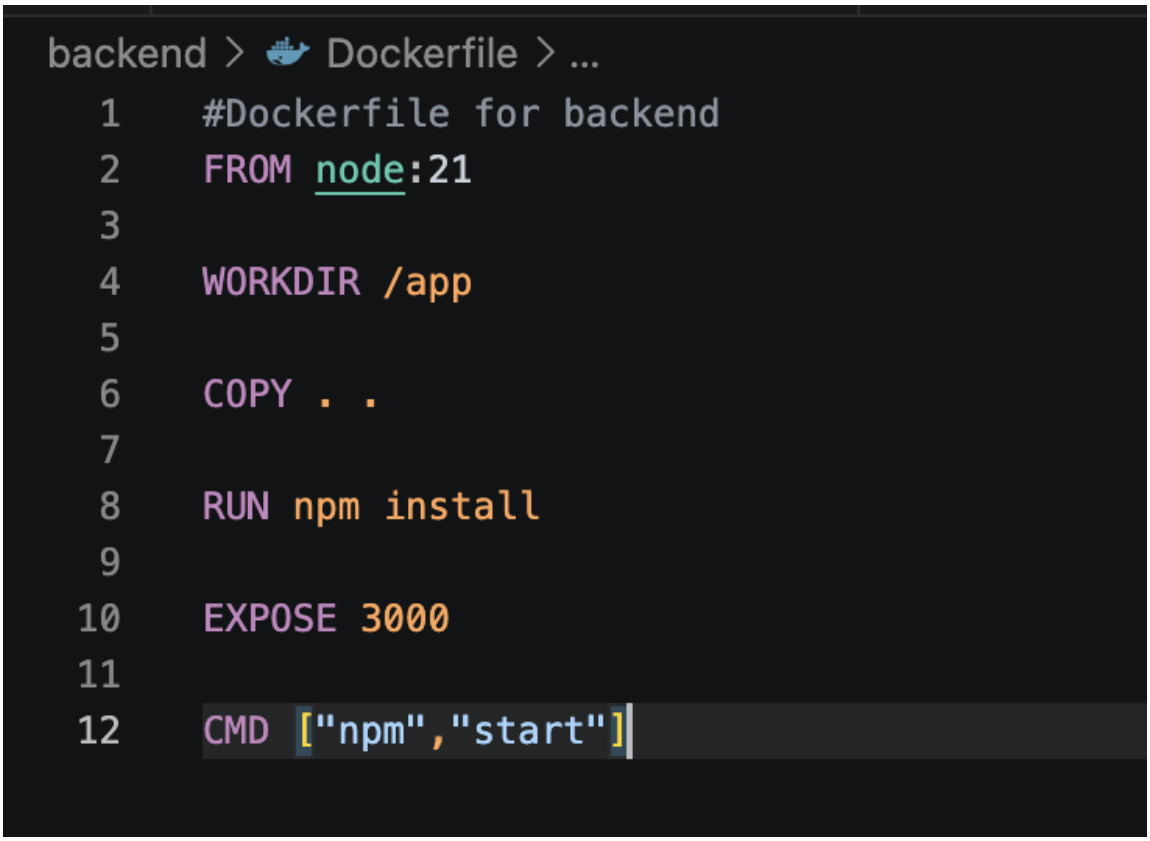

Figure 9 — Backend Dockerfile.

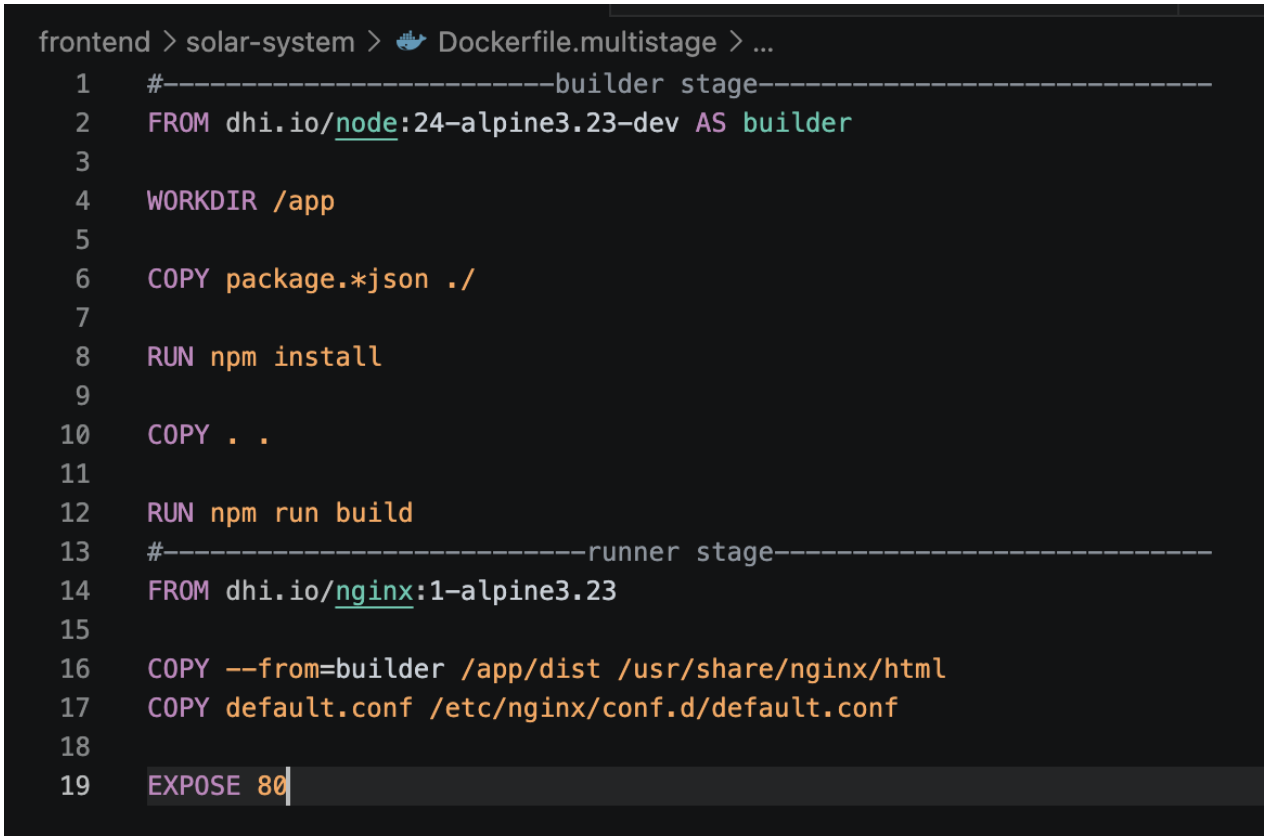

Figure 10 — Solar System multi-stage Dockerfile.

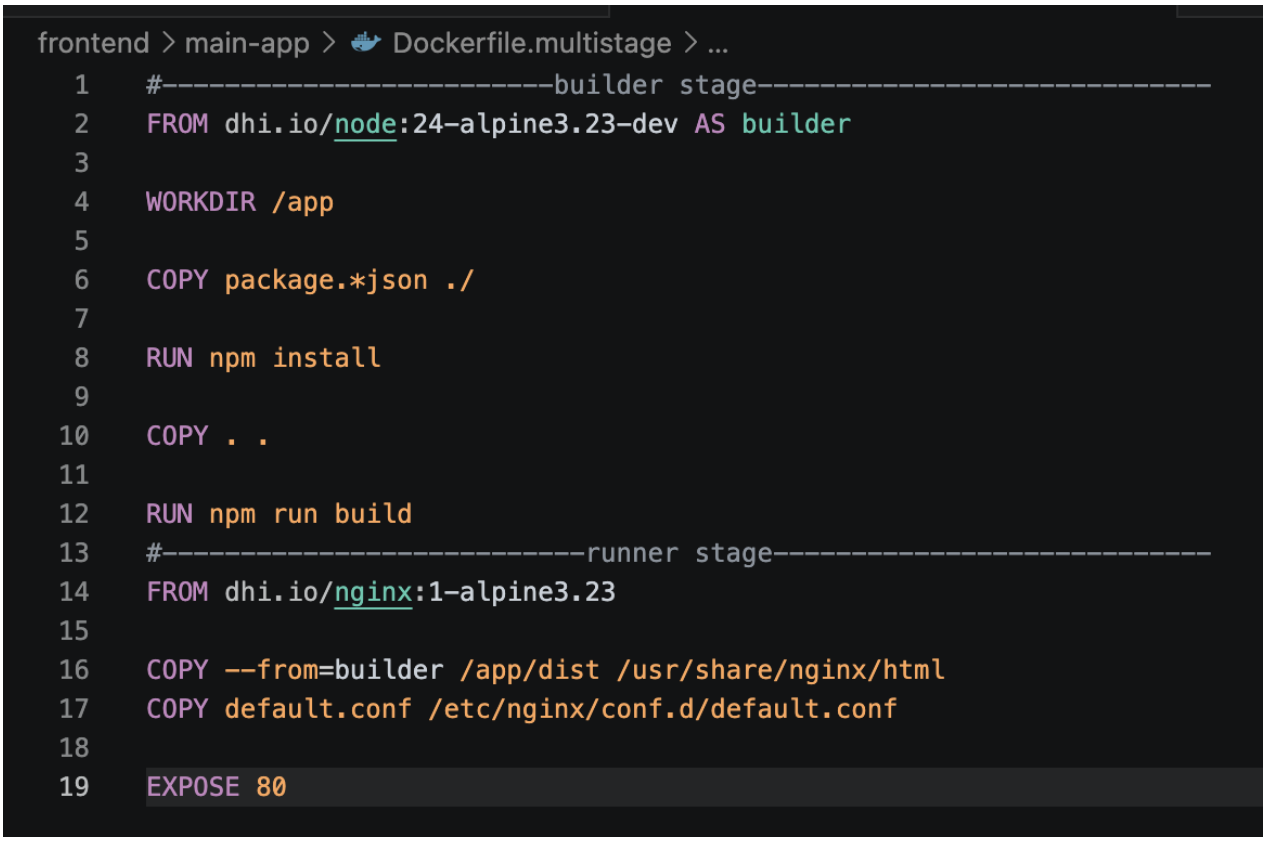

Figure 11 — Main App multi-stage Dockerfile.

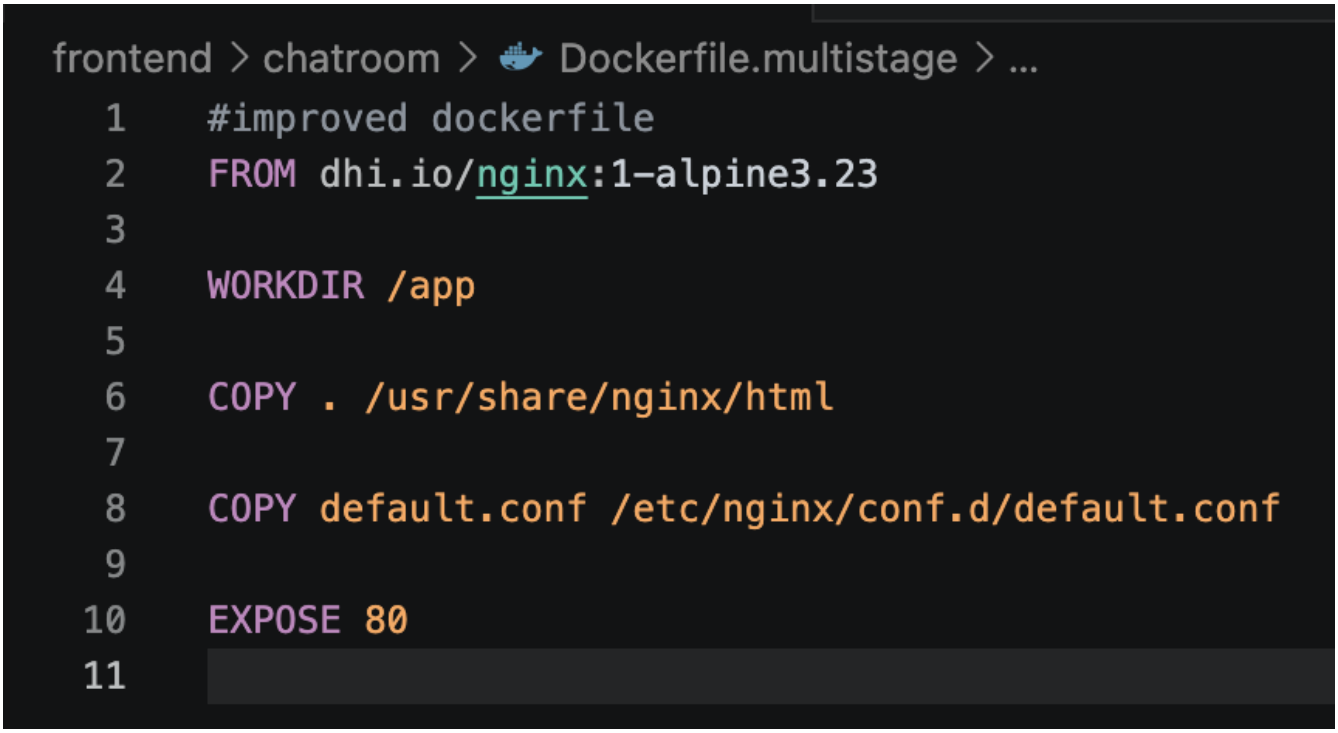

Figure 12 — Chatroom multi-stage Dockerfile.

backend > Dockerfile.multistage > ...

```dockerfile
#-------------------------builder stage------------------------------
FROM dhi.io/node:24-alpine3.23-dev AS builder

WORKDIR /app

COPY package.*json ./

RUN npm install --omit=dev

#-------------------------runner stage-------------------------------
FROM  dhi.io/node:24-alpine3.23

WORKDIR /app

ENV NODE_ENV=production

COPY --from=builder /app/node_modules ./node_modules
COPY package.json ./
COPY src ./src

EXPOSE 3000

CMD ["node", "src/server.js"]
```

Figure 13 — Backend multi-stage Dockerfile.

STEP 4

Docker Image Optimization

Multi-stage builds reduced the final runtime image sizes by separating build dependencies from production runtime layers.

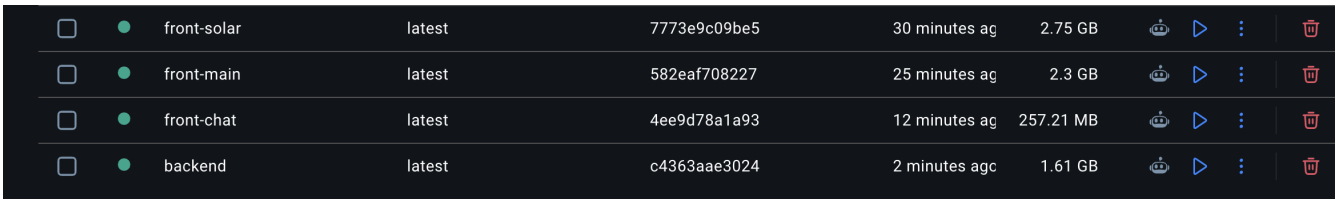

Figure 14 — Docker image sizes after optimization/building.

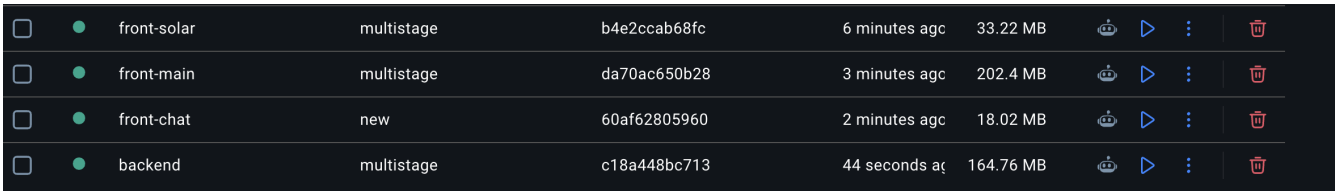

Figure 15 — Docker image size comparison evidence.

Observed optimized sizes: front-solar ≈ 33 MB • front-main ≈ 202 MB • front-chat ≈ 18 MB • backend ≈ 165 MB.

STEP 5

Docker Compose & Networking

Docker Compose coordinates the application services and connects them through the project Docker network. The Compose file also defines database health checking, dependencies, volumes, and service configuration.

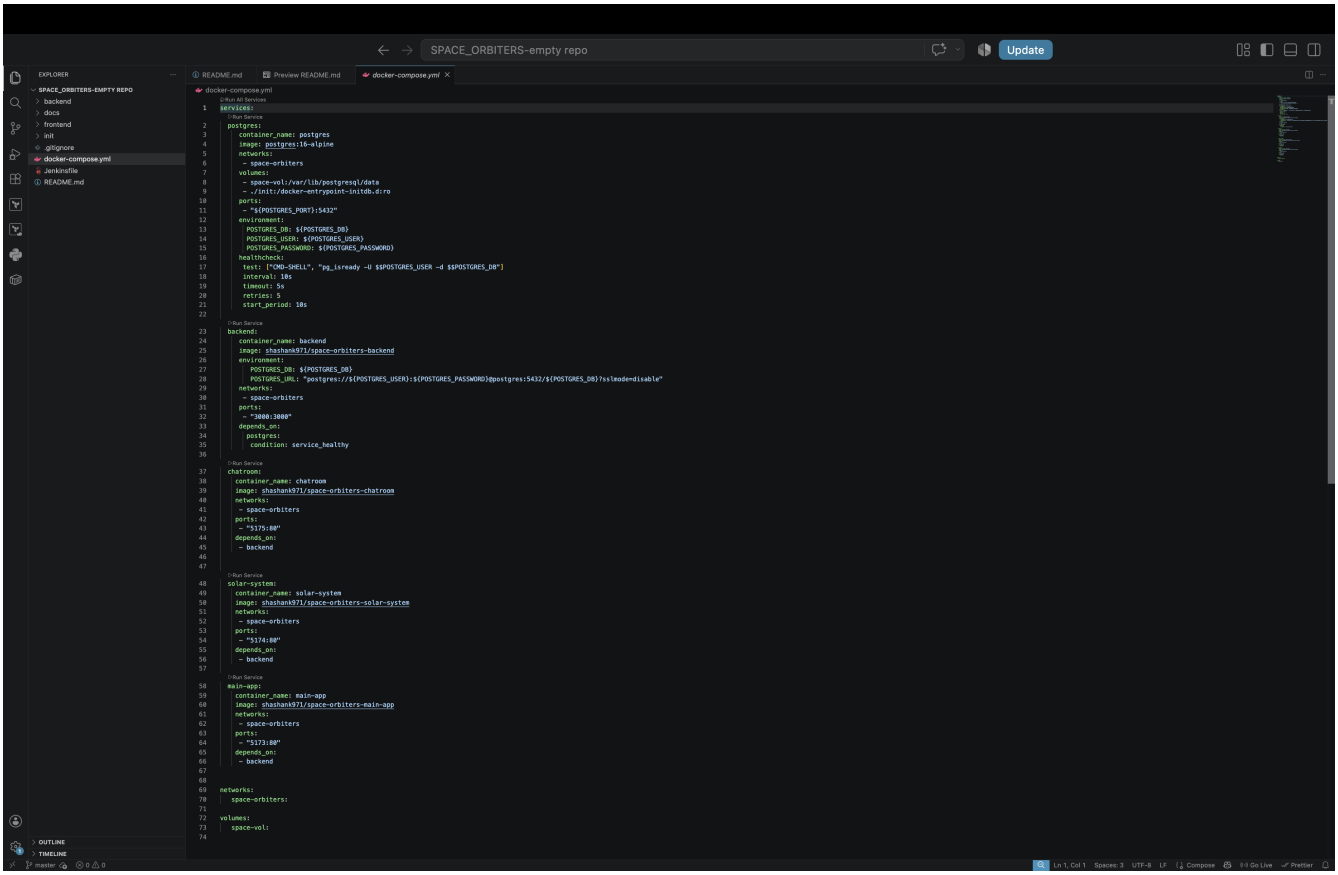

Figure 16 — Docker Compose configuration showing services, network, PostgreSQL volume, healthcheck, and dependencies.

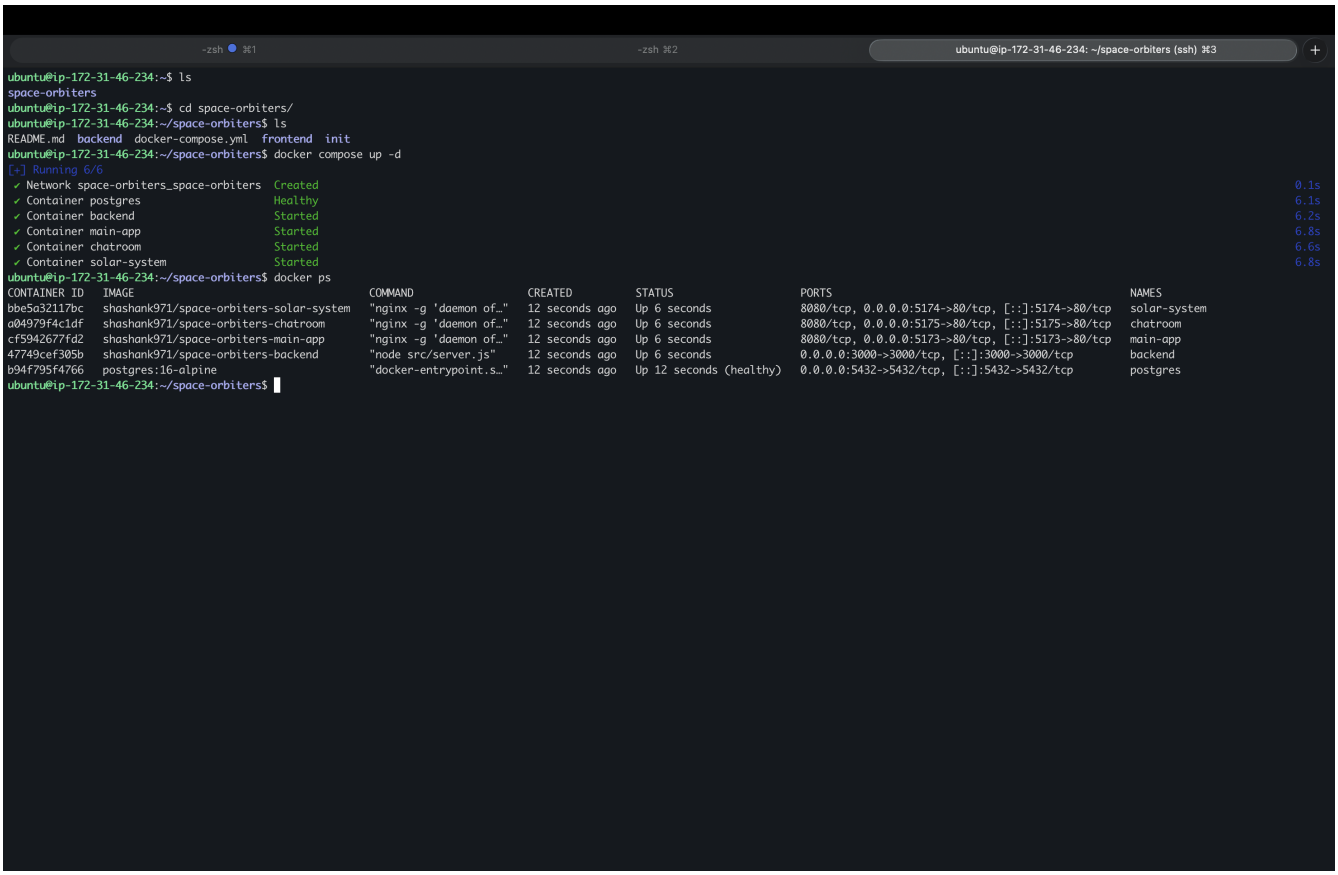

Figure 17 — Compose startup and running-container verification on the EC2 environment.

STEP 6

Persistent Database Data

PostgreSQL is used as the persistent data tier. Database initialization is supplied through SQL scripts and PostgreSQL data is stored in the Docker volume used by the Compose deployment.

Persistence evidence: The supplied demonstration video contains the persistence verification, showing the database/data workflow used for the project.

Open the Google Drive persistence-proof video

The video link is intentionally included inside the persistence section as evidence for this requirement; no separate project-demo section is added.

STEP 7

AWS EC2 Deployment

The containerized application was deployed on an AWS EC2 Ubuntu instance. The security-group evidence documents the network access configured for the deployed environment.

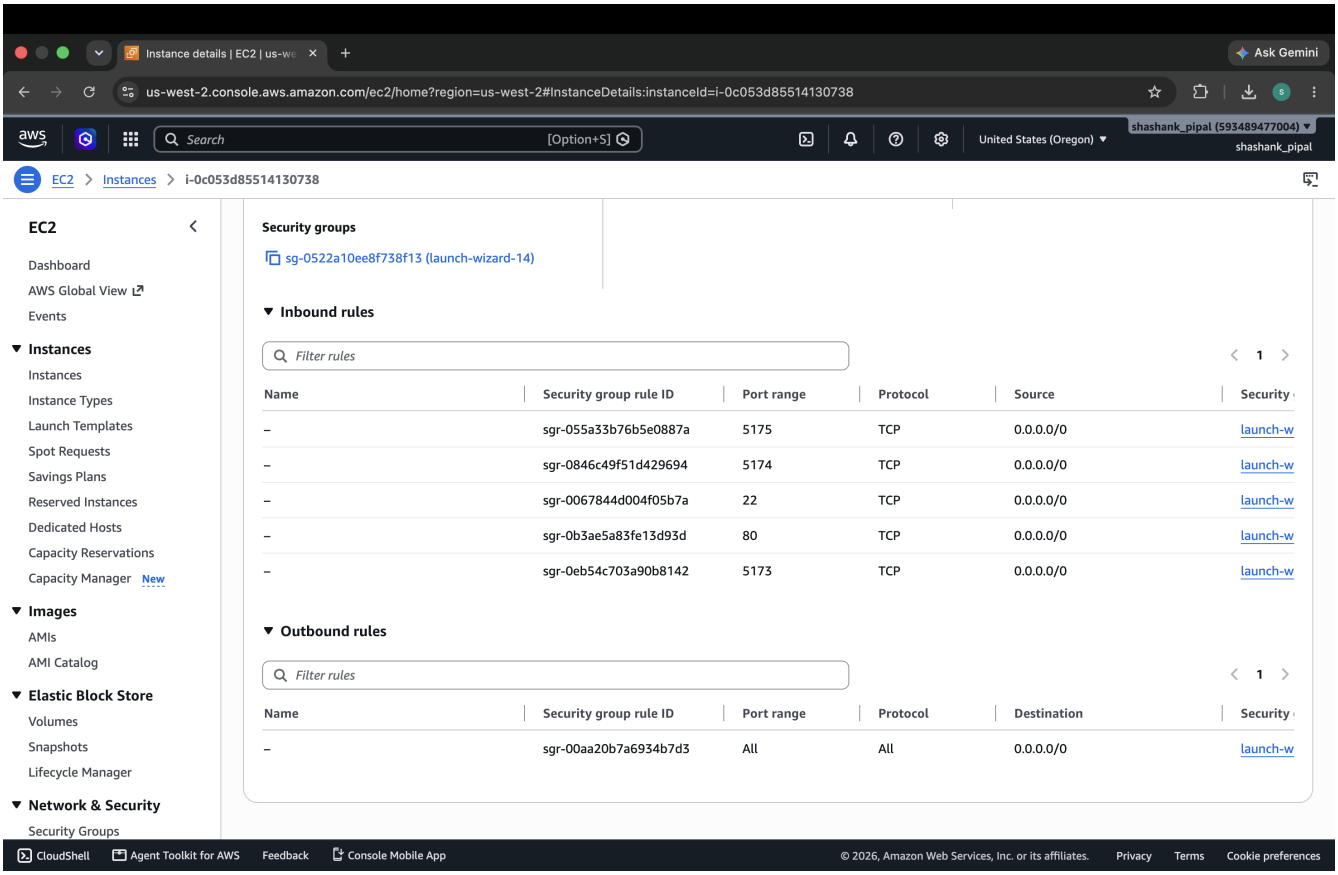

Figure 18 — EC2 security-group inbound rules used during deployment.

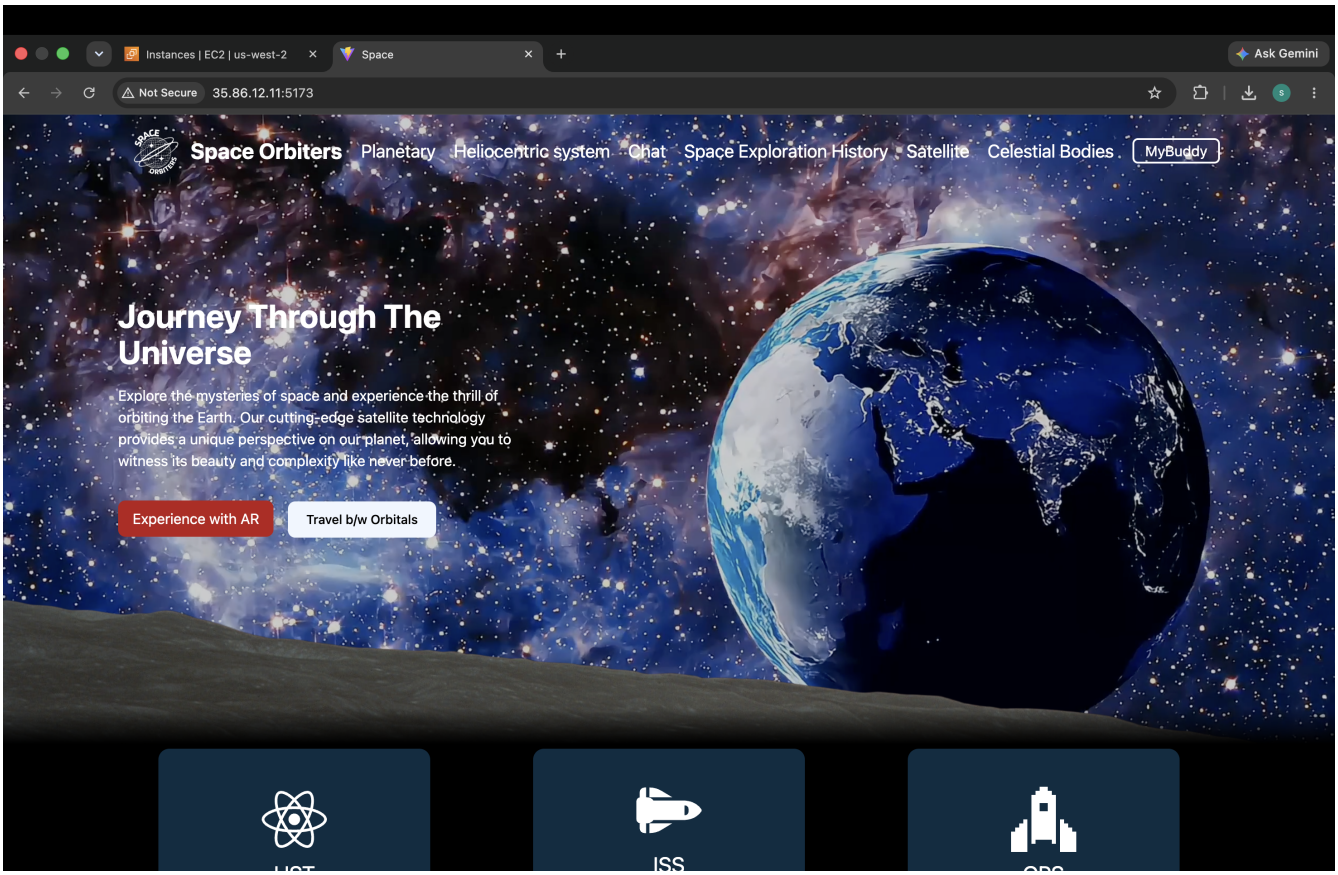

Figure 19 — Main Space Orbiters application running.

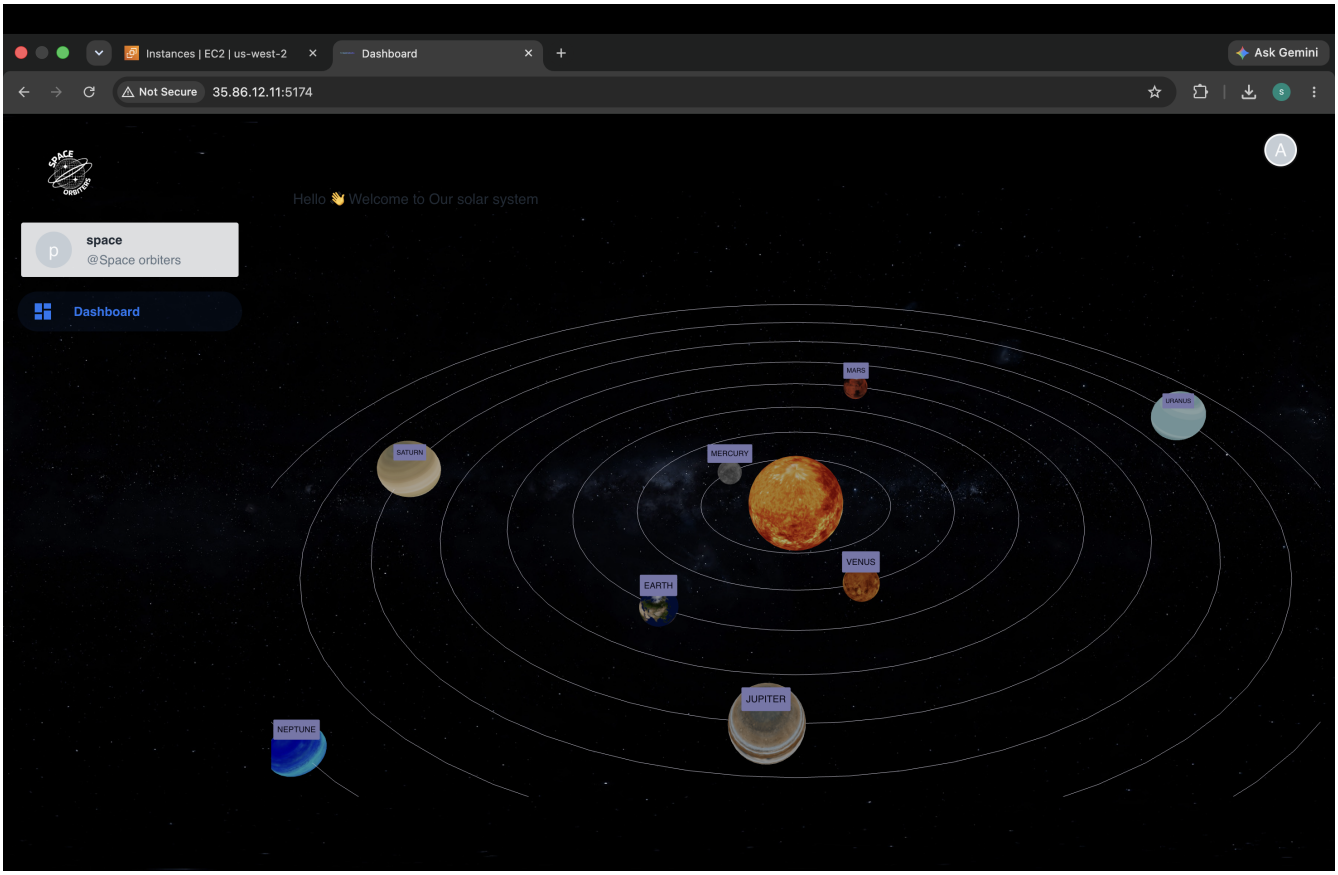

Figure 20 — Solar System application running.

STEP A

Additional Enhancement — Elastic IP & DNS

A stable public address was associated with the EC2 instance, and the GoDaddy DNS record was configured so the application can be reached through a named subdomain instead of an EC2 IP address.

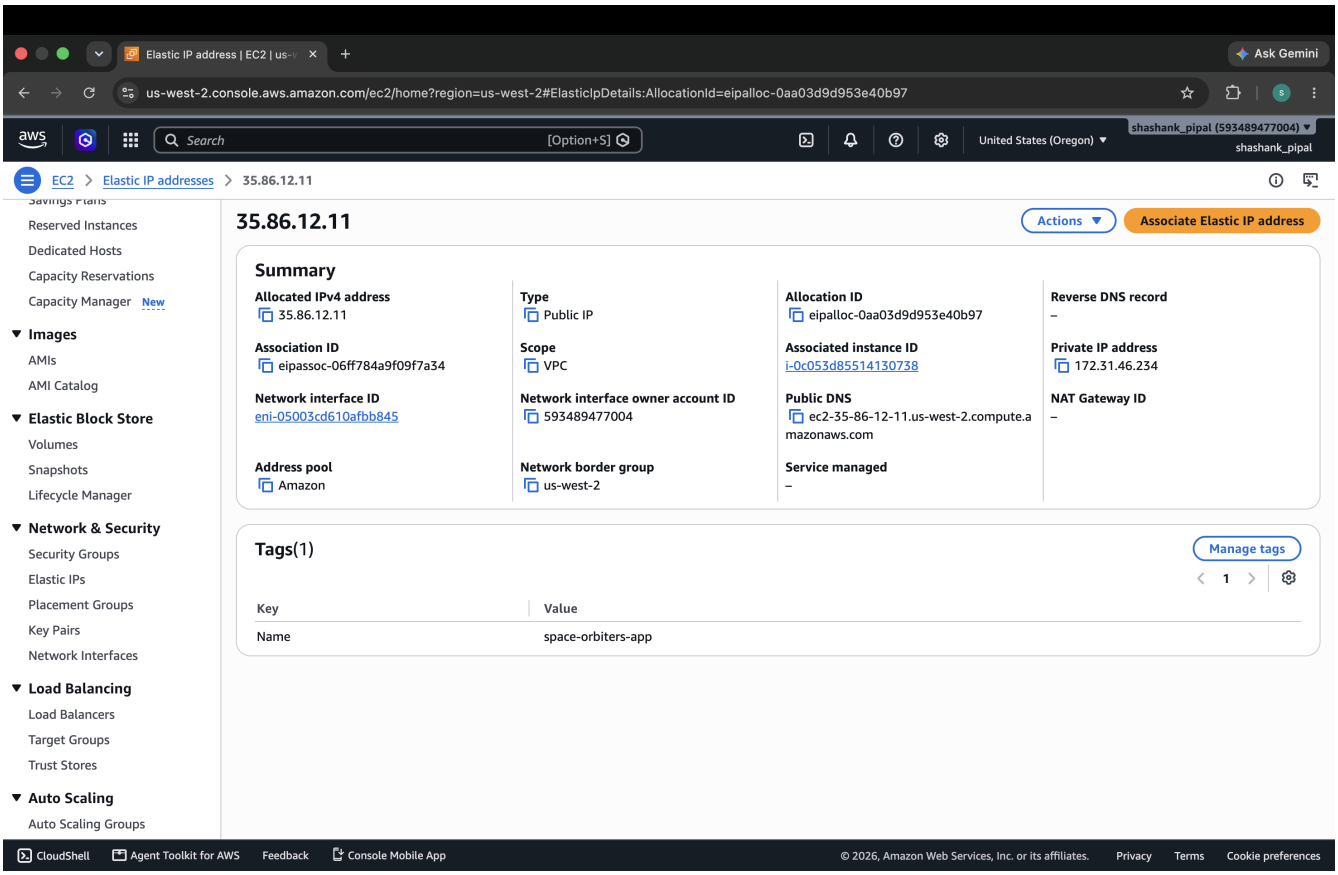

Figure 21 — AWS Elastic IP 35.86.12.11 associated with the EC2 instance.

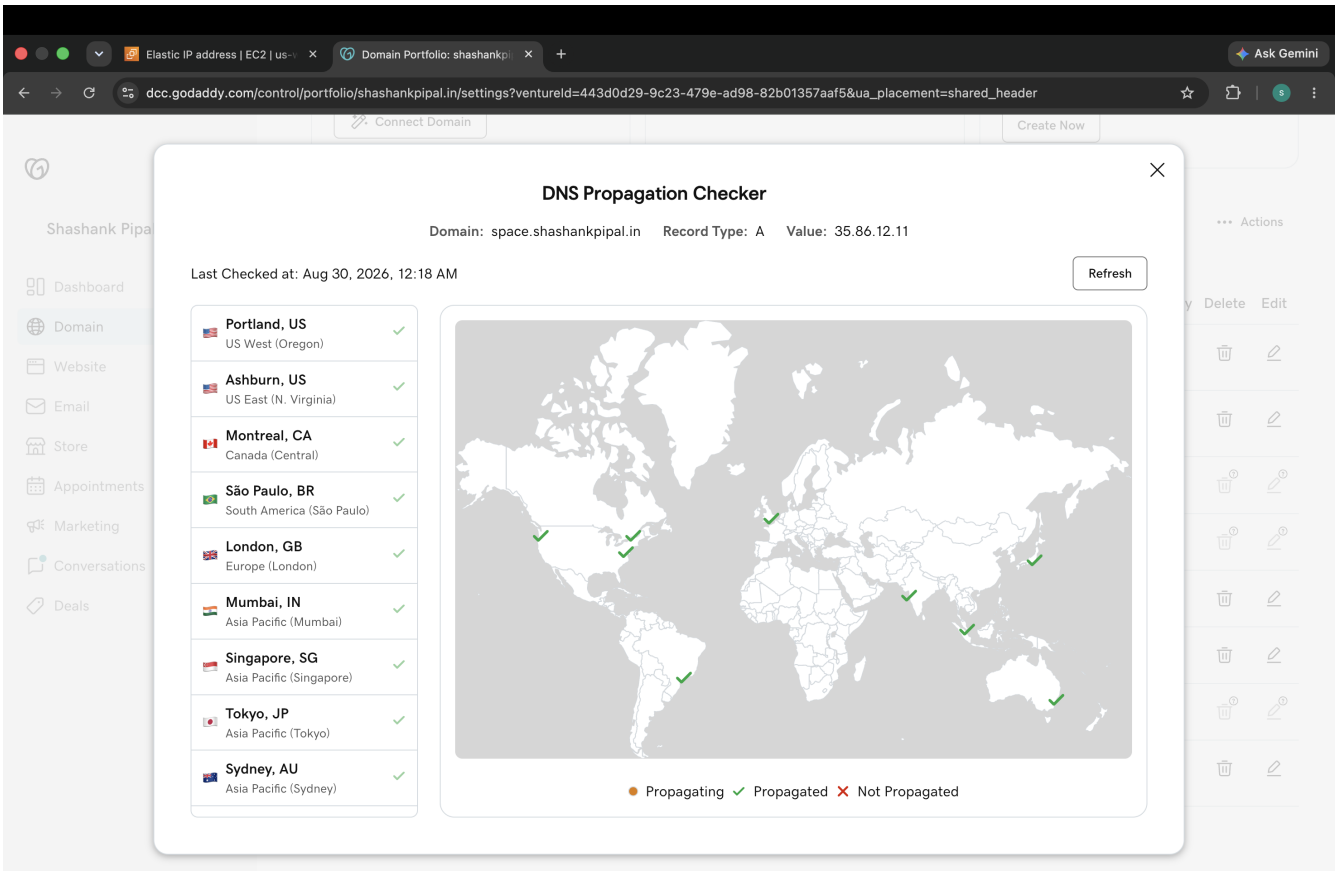

Figure 22 — GoDaddy DNS propagation check for space.shashankpipal.in pointing to 35.86.12.11.

STEP B

Additional Enhancement — Nginx Reverse Proxy & HTTPS

Nginx was introduced as the public reverse proxy. It handles HTTPS and routes the application by path, while Certbot/Let's Encrypt provides the TLS certificate.

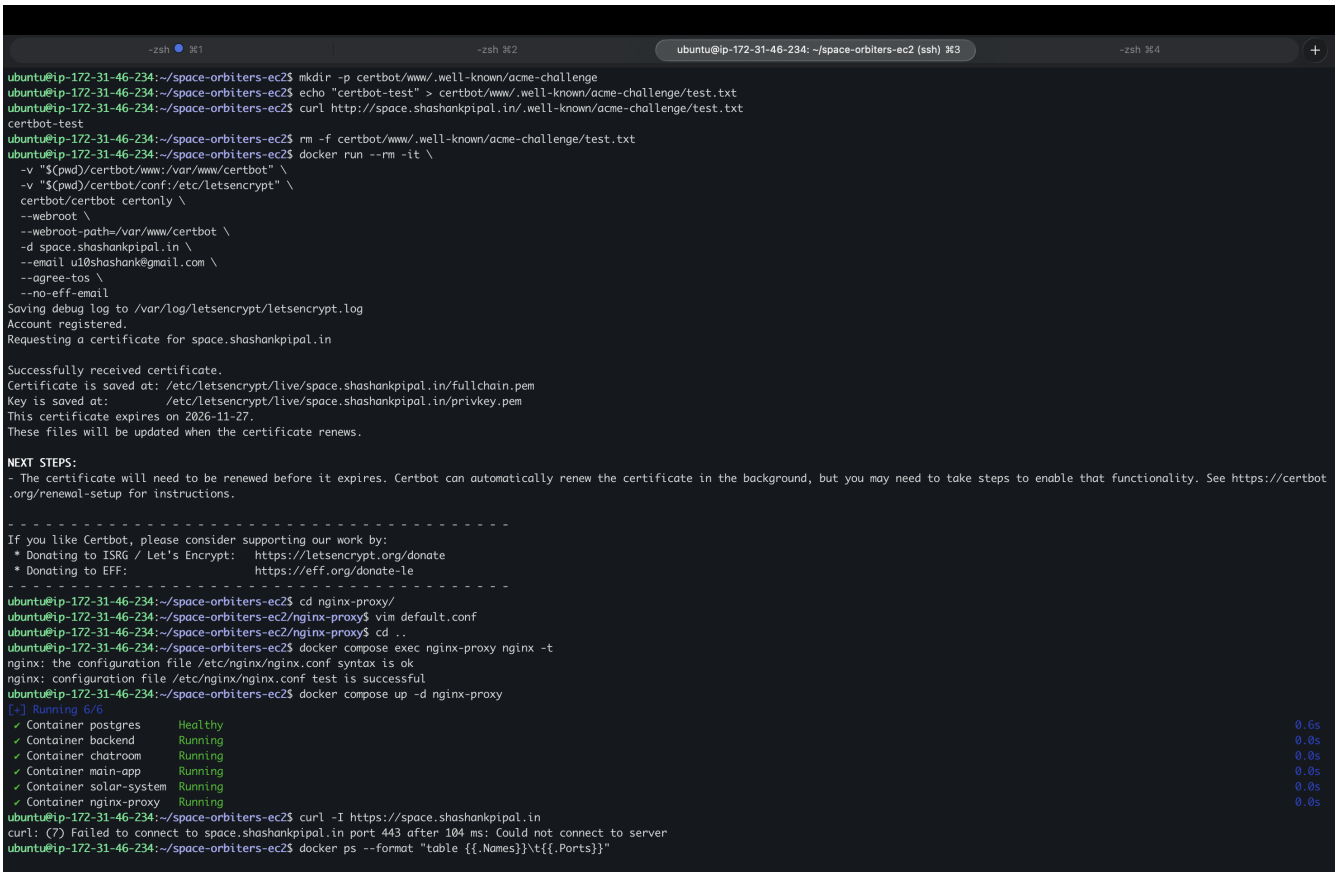

Figure 23 — Certbot successfully obtaining the Let's Encrypt certificate for space.shashankpipal.in.

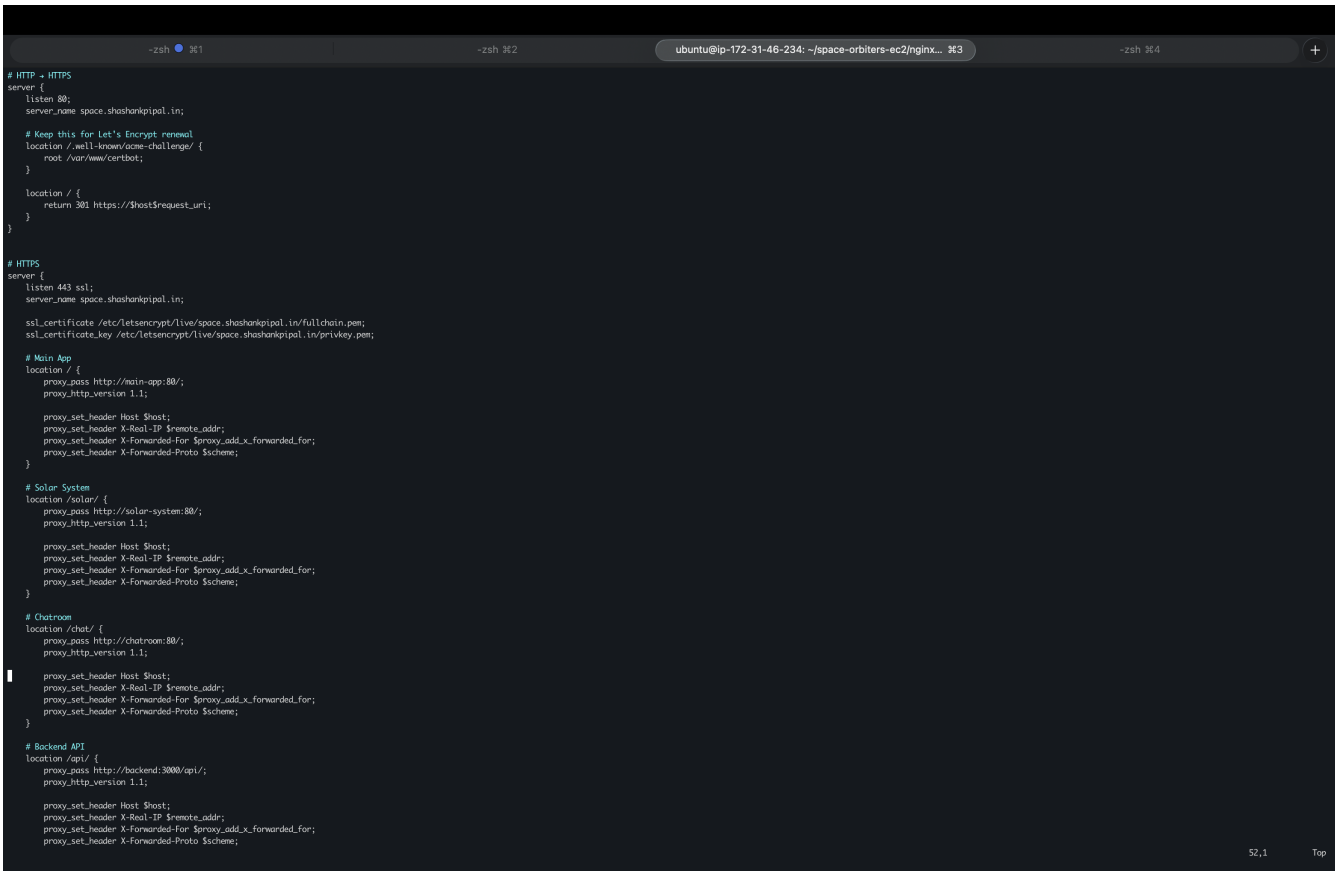

Figure 24 — Nginx HTTP→HTTPS redirect, TLS configuration, and path-based reverse-proxy routes.

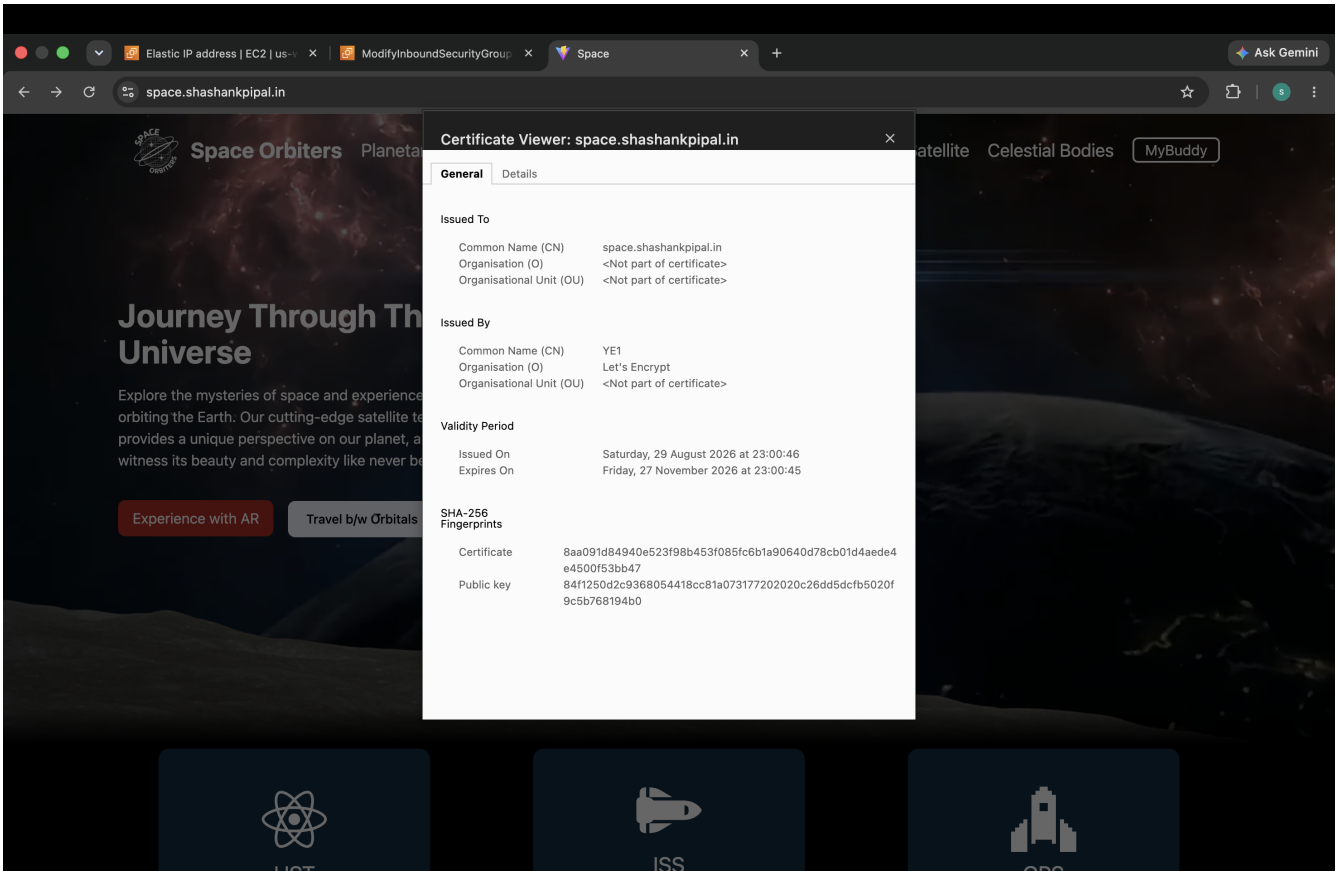

Figure 25 — Browser certificate viewer showing the Let's Encrypt certificate issued to space.shashankpipal.in.

STEP C

Additional Enhancement — Jenkins CI/CD & DevSecOps

Jenkins automation was prepared using a reusable Shared Library. The pipeline is designed around source checkout, code quality, dependency/security scanning, Docker build and image scanning, Docker Hub publishing, and deployment.

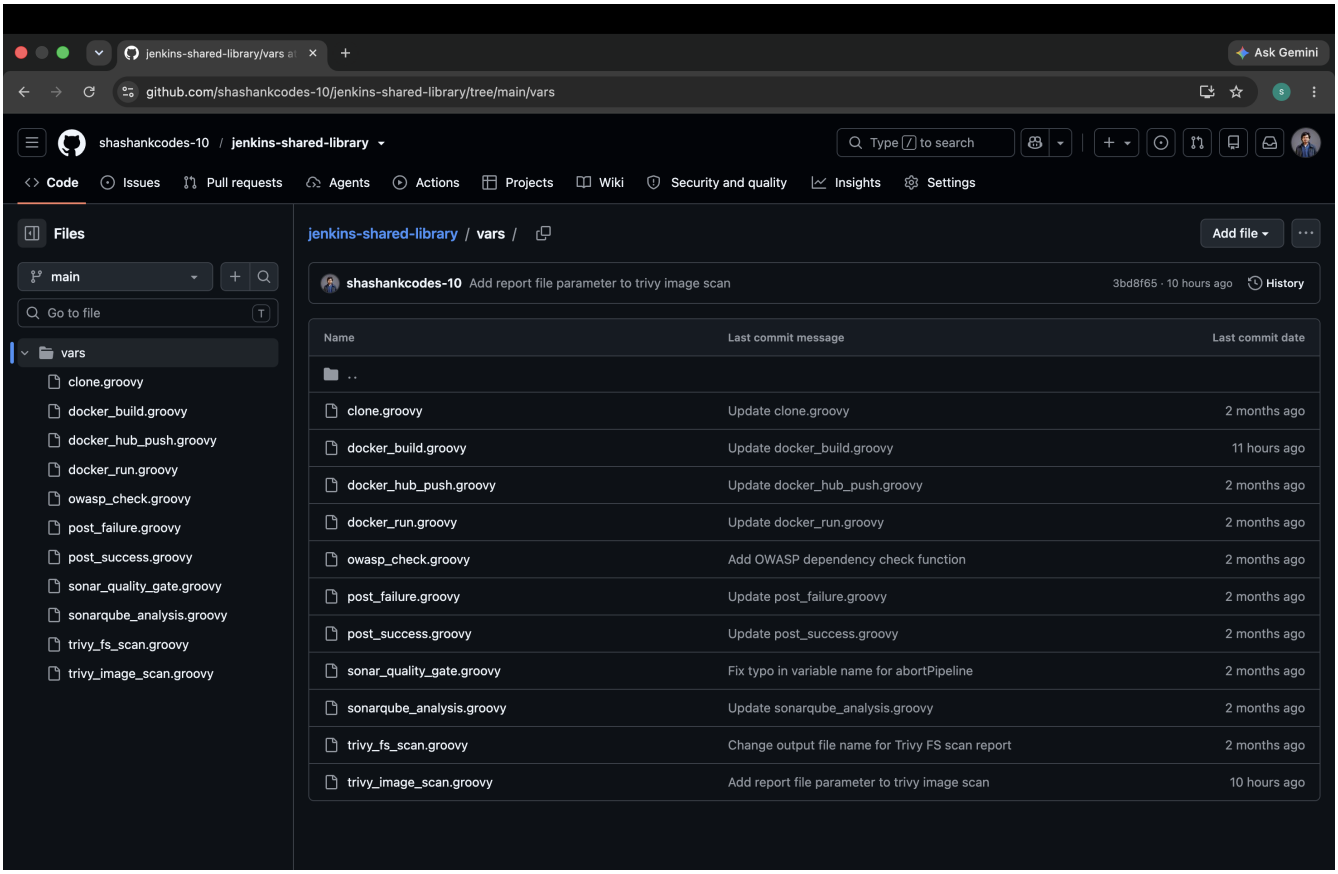

Figure 26 — Jenkins Shared Library vars directory containing reusable pipeline steps.

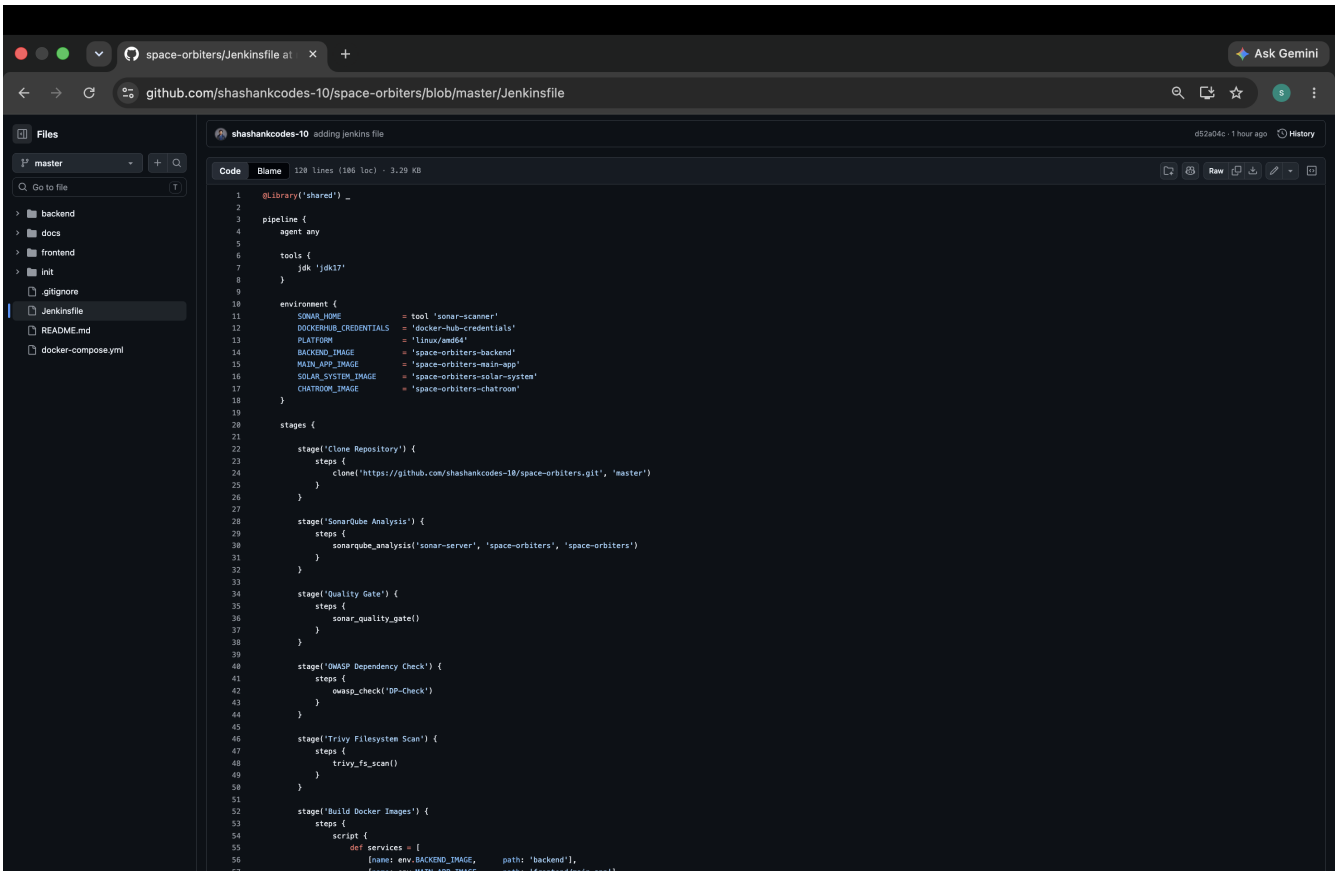

Figure 27 — Jenkinsfile for the Space Orbiters CI/CD pipeline.

Implementation Summary

The project was transformed into a containerized 3-tier application with PostgreSQL persistence and an AWS EC2 deployment. The implementation includes Docker and Docker Compose, multi-stage image optimization, Git/GitHub practices, and a structured service architecture.

Beyond the core project work, the deployment was extended with an Elastic IP, GoDaddy DNS, Nginx reverse proxy, Let's Encrypt HTTPS, and a Jenkins-based DevSecOps pipeline design.

Core project	Implemented and evidenced through the main steps of this report.
Persistent data	Evidence supplied through the linked Google Drive persistence video in Step 6.
Production enhancements	DNS, Elastic IP, Nginx, HTTPS/SSL, Certbot, and Jenkins are explicitly separated from the core requirements.

End of Report